



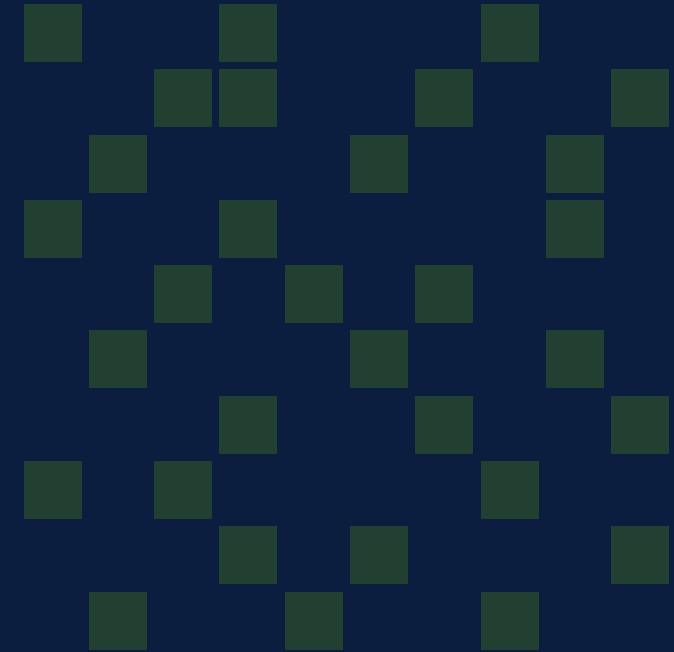
Faculty of Computing
University of Sri Jayewardenepura

Conway's Game of Life

Sequential CPU vs. Parallel CUDA GPU Implementation

CCS3032 — HIGH-PERFORMANCE COMPUTING
B.Comp (Hons.) in Computer Science

M.A.O. Rodrigo | Reg. No: FC115278 | Index No: FC221019 | Group 2



What we'll cover

01 Problem & Objectives

Why Game of Life is a natural HPC benchmark

02 Experimental Setup

Platform, grid parameters, benchmarking method

03 CPU Implementation

Sequential 2D-array baseline in C++

04 CUDA Implementation

Thread-per-cell parallel GPU kernel

05 Correctness & Results

Verification, timings, ~440x speedup

06 Conclusion

What the results teach us about HPC

The problem: a grid that never stops recomputing

Conway's Game of Life is a cellular automaton: every cell in an $n \times n$ grid is alive or dead, and the whole grid evolves in discrete generations under one fixed rule based on each cell's 8 neighbours.

Every cell must be evaluated independently, every generation — an ideal candidate for High-Performance Computing techniques.

 **< 2 neighbours**
Underpopulation

Dies

 **2 or 3 neighbours**
Stable

Survives

 **Dead + 3 neighbours**
Reproduction

Becomes alive

COMPUTATIONAL COST

$O(n^2)$ per generation

Every one of the n^2 cells needs an 8-neighbour scan, every single generation.

For g generations:

$O(n^2 \times g)$

*This project's grid: $1024 \times 1024 = 1,048,576$
cells $\times$ 100 generations $\approx$ 105 million cell
evaluations per run.*

Objectives

- | | | |
|---|-----------------------------|---|
| 1 | Implement | Build the same 1024×1024 simulation two ways — sequential C++ 2D arrays, and CUDA with one thread per cell. |
| 2 | Keep it fair | Identical seed, density and rules in both versions, so results are directly comparable. |
| 3 | Benchmark rigorously | Untimed warm-up pass, then repeated timed trials — report mean and range, not a single lucky run. |
| 4 | Verify correctness | Confirm both implementations reach identical simulation states, generation by generation. |
| 5 | Explain the speedup | Identify the specific GPU techniques responsible for the performance gained. |

Platform: Google Colab, same host, two engines



CPU (Baseline)

Model	Intel(R) Xeon(R) @ 2.20GHz
Socket(s)	1
Core(s)/socket	1
Threads/core	2
CPU(s) online	0, 1
Compiler flags	g++ -O2 -std=c++17



GPU (Parallel)

Model	NVIDIA Tesla T4
Compute capability	7.5
Total memory	15,360 MiB
Driver version	580.82.07
Block size	16 × 16 (256 threads)
Compiler flags	nvcc -O3

Same grid, same rules, same starting point

1024×1024

Grid size (1,048,576
cells)

100

Generations per trial

~10%

Initial live-cell density

srand(42)

Fixed seed — identical in
both

RULES: STANDARD CONWAY B3/S23 — APPLIED IDENTICALLY ON BOTH SIDES

Because both engines start from the exact same random grid and apply the exact same rule, **any difference measured later is purely a difference in execution speed — not in what was computed.**

Sequential baseline: one core, one cell at a time

● Flat contiguous array

`std::vector<int>` indexed as `y*WIDTH+x` instead of nested 2D arrays — far more cache-friendly for large grids.

● 8-neighbour Moore scan

`countAliveNeighbors()` checks all 8 neighbours with boundary checks at the grid edge.

● Double-buffered update

Each generation writes into a second buffer, then `current.swap(next)` in $O(1)$ — avoids a cell reading another cell's already-updated state.

● Compiler optimization

Compiled with `g++ -O2 -std=c++17` for loop unrolling & auto-vectorization — the only parallelism available to a single core.

nextState() — the rule

```
if (current==1 && n<2) return 0;
if (current==1 && (n==2||n==3))
    return 1;
if (current==1 && n>3) return 0;
if (current==0 && n==3) return 1;
return current;
```

`idx(x,y) = y * WIDTH + x`

`current.swap(next);`

Single execution model: no thread ever runs — all 1.05M cells are visited by the same core, one after another.

Parallel kernel: one GPU thread per cell

● Thread-per-cell mapping

Each of the $\sim 1.05\text{M}$ cells gets its own GPU thread computing (x,y) from block/thread indices — all evaluated simultaneously.

● Warp-aligned 16×16 blocks

256 threads per block, a multiple of the 32-thread warp size — no wasted, partially-filled warps.

● Grid never leaves the GPU

d_current / d_next swapped by pointer after every kernel launch — zero host $\leftrightarrow$ device transfer inside the timed loop.

● Error-checked launches

Every CUDA API call wrapped in a CUDA_CHECK macro, since CUDA errors otherwise fail silently.

The whole kernel launch

```
dim3 blockDim(16, 16);
dim3 gridDim(
    (W+15)/16, (H+15)/16);

for (gen = 0; gen < 100; gen++) {
    kernel<<<gridDim, blockDim>>>
        (d_current, d_next, W, H);
    std::swap(d_current, d_next);
}
```

Same rule, same logic — only the execution model changes. That's what makes the CPU/GPU comparison meaningful.

How the timing was made trustworthy

- 1 warm-up pass**

Full untimed 100-generation run first — primes memory allocation, CPU caches, and the GPU context/driver.
- 2 5 timed repeats**

Grid reset to the identical initial state before every repeat; `std::chrono::high_resolution_clock` measures wall-clock time.
- 3 `cudaDeviceSynchronize()`**

Brackets GPU timing so every kernel has actually finished before the clock stops — no async under-counting.
- 4 Mean & range reported**

Not a single number — mean and min-max range across the 5 repeats, to reveal variability, not hide it.
- 5 Frame-saving kept separate**

GIF/PGM output only runs with `saveFrames = true`, outside the timed runs, so disk I/O never distorts the comparison.

Correctness first: same simulation, same states

Alive-cell counts were extracted from saved frames at matching generations on both implementations.

The counts were identical at every checkpoint — confirming the GPU is computing the same simulation, not just a faster one.

```

from PIL import Image
from IPython.display import Image as IPImage

img = Image.open("frames/frame_0098.pgm").resize((256, 256), Image.NEAREST)
img.save("cpu_frame_98.png")
IPImage("cpu_frame_98.png")

```



CPU — 98th frame

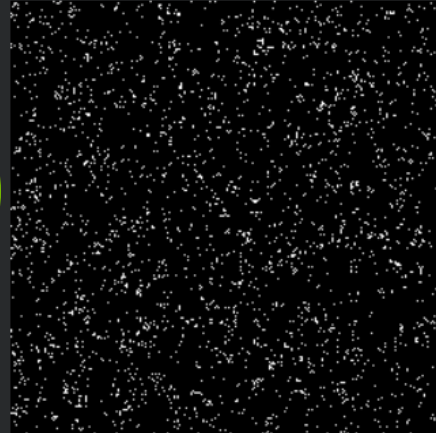


```

from PIL import Image
from IPython.display import Image as IPImage

img = Image.open("frames_cuda/frame_0098.pgm").resize((256, 256), Image.NEAREST)
img.save("cuda_frame_98.png")
IPImage("cuda_frame_98.png")

```



GPU — 98th frame

RESULTS

Measured execution times (5 repeats, after warm-up)

	CPU (2D array)	CUDA (GPU)
Repeat 1	2404.47 ms	6.41658 ms
Repeat 2	2874.96 ms	6.40737 ms
Repeat 3	3389.84 ms	6.40829 ms
Repeat 4	3003.92 ms	6.41657 ms
Repeat 5	2423.61 ms	6.40888 ms
Mean total	2819.36 ms	6.41154 ms
Range (min–max)	2404.47 – 3389.84 ms	6.40737 – 6.41658 ms
Mean / generation	28.19 ms	0.0641 ms

SPEEDUP

≈ 440×

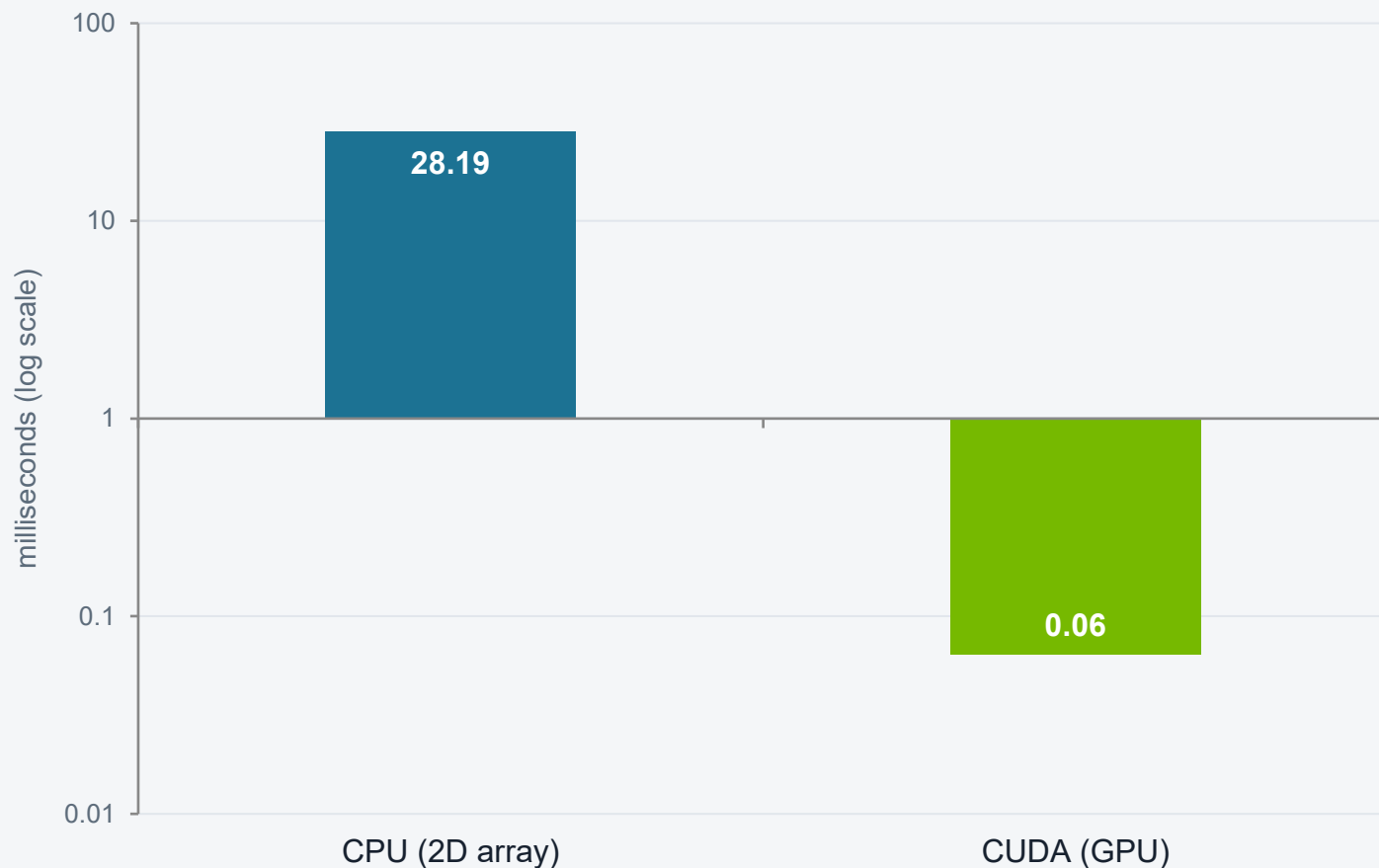
faster on GPU

Speedup = CPU time ÷ GPU time
= 2819.36 ÷ 6.41154
= 439.73 ≈ 440

*100-generation, 1024×1024
simulation*

RESULTS

Mean time per generation — log scale



Stability

GPU repeats clustered within a 0.01 ms band — essentially deterministic once warmed up.

Variance

CPU repeats spread 2404–3390 ms (~41%), consistent with shared-VM contention on Colab.

Scale gap

0.0641 ms/gen on GPU vs 28.19 ms/gen on CPU — a log scale is needed just to show both bars.

Speed isn't the only story — predictability matters too

CPU: shared, noisy infrastructure

- Repeats 1 & 5 sit near 2400–2420 ms
- Repeats 2–4 elevated, peaking at 3389.84 ms
- That's ~41% higher than the fastest repeat
- Warm-up already covers first-touch allocation & cache priming
- Most likely cause: contention on Colab's shared, non-dedicated CPU — not the algorithm itself

GPU: isolated, deterministic execution

- All 5 repeats fall within a 0.01 ms band
- 6.40737 – 6.41658 ms — essentially indistinguishable
- Grid stays resident in device memory the whole loop
- Each kernel runs in isolation on dedicated SMs, not dependent on host OS scheduling
- Consistency is itself a meaningful HPC result, not just a side effect

CONCLUSION

What made the GPU ~440x faster

1 Thread-per-cell parallelism

~1.05M independent threads replace ~1.05M sequential loop iterations.

2 Warp-aligned 16×16 blocks

256 threads/block — a clean multiple of the 32-thread warp, no wasted warps.

3 Zero host↔device transfer in the loop

Buffers swapped by pointer on-device; transfer overhead never enters the timed path.

TAKEAWAY

Game of Life is an embarrassingly parallel, computationally expensive problem — exactly the case HPC techniques exist for. It's impractical to push much further on one CPU core, but it maps naturally onto thousands of independent GPU threads, verified correct and measured at ~440x, with the GPU also delivering far more predictable execution.

Thank you

M.A.O. Rodrigo | Reg. No: FC115278 | Index No: FC221019

CCS3032 — High Performance Computing